

$$= n = O(n^2)$$

$$(2) \quad n^2 \cdot 2^{3 \log_2 n} = O(n^5)$$

$$= n^2 \cdot n^{3 \log_2 2}$$

$$= n^2 \cdot n^3$$

$$= n^5$$

$$= n^5 = O(n^5)$$

$$(3) \quad 2^n = 2^{2^n}$$

$$2^n = 2^n \cdot 2^n$$

$$2^n \leq 2^{2^n}$$

$$2^n = O(2^{2^n}) \text{ True}$$

$$(4) \quad \log n > \log \log n$$

$$\log n \neq O(\log n)$$

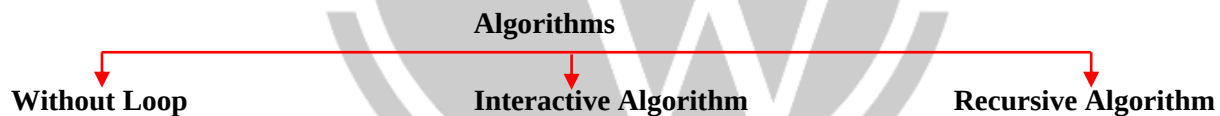
False

$$(5) \quad \log \log n \leq n \log n$$

$$\log \log n = O(n \log n)$$

True

1.8. Analysis of an Algorithm



1.8.1 Without loop

Example:

```

int fun (in + n)
{
    return n*(n+1)/2;
}
  
```

Solution.

Here 1 multiply, 1 division, 1 addition

$\therefore O(1)$ [no loops, no recursion]

1.8.2. Iterative Algorithm Analysis

Example 1:

```

for (i = 1; i ≤ n; i = i * 2)
    printf("Sushil")
  
```

Solution.

$$i=1, 2, 2^2, 2^3 \dots, 2^k$$

→

$$2k \leq n$$

$$k \log 2 \leq \log n$$

$$k \leq \frac{\log n}{\log 2}$$

$$\therefore k \leq \log_2 n$$

$$k = \lfloor \log_2 n \rfloor$$

So, this will execute $\lfloor \log_2 n \rfloor + 1$ time and Complexity $O(\log_2 n)$

Example 2:

```
For (i=1; i ≤ n; i=i*3)
printf("Aaveg");
```

Solution.

So, this will execute $\lfloor \log_3 n \rfloor + 1$ time and complexity $O(\log_3 n)$

➤ $i = 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow \dots \rightarrow n$
 $i = n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots 1$

Example 3:

```
for (i = 1; i ≤ n; i++)
{
    for (j=1; j ≤ 10; j++)
    {
        printf("Dhananjay");
    }
}
```

Solution.

This will execute $10 \cdot n$ times and complexity $O(n)$

Example 4:

```
for (i = 1; i ≤ n; i = i*3)
    for (j = 1; j ≤ n; j++)
        printf("Prapti");
```

Solution.

Total $n(\lfloor \log_3 n \rfloor + 1)$ time execute and Complexity = $O(n \log_3 n)$

1.8.3. Recursive Algorithm Analysis

Example 1:

```
void fun (in + n)    T(n)
{
    if (n > 0)        1 compare; C1 time
    {
        if ("% d", n); ← C2 time
        fun (n - 1); ← T(n-1)
    }
}
```

Let $T(n)$ be the Complexity time taken by algo for n size i/p

Solution.

$$T(n) = C_1 + C_2 + T(n-1)$$

$$T(n) = T(n-1) + C$$

$n > 0$

$$T(0) = C$$

Constant

$$\begin{aligned} T(n) &= C; & n &= 0 \\ T(n) &= T(n-1) + C; & n &> 0 \end{aligned}$$

Example 2:

```
void fun (in + n)    T(n)
{
    if (n > 0) ← C1 time
    {
        for (i = 1; i <= n; i + 1) ← n time
        printf("Hello");
        fun (n - 1); ← T(n - 1)
    }
}
```

Solution.

$$T(n) = C_1 + n - 1 + T(n-1)$$

$$= T(n-1) + n$$

$$T(0) = C$$

$n > 0$

$n = 0$

Example 3:

```
void fun (in + n)    T(n)
{
    if (n > 0) ← C1
    {
        for (i = 1; i <= n; i = i*2) ← ⌊ log2 n ⌋
    }
}
```